Confidential VM上の機密情報拡散追跡機能における追跡ログの由来確認と改ざん検知機構の実現と評価

乾 朱来[1]　山内 利宏[2]

概要：Confidential Virtual Machine（以降，CVM）は処理中のデータを保護できる一方，CVM にログインできない主体（以降，外部の主体）は，CVM を運用する主体（以降，ゲストオーナー）によるログのすり替えや改ざんを検知できない．そこで，我々は，CVM 内で機密情報拡散追跡機能が生成するログを対象として，外部からログの由来確認と改ざん検知を可能にする機構（以降，提案機構）を提案した．提案機構は，AMD SEV-SNP の Virtual Machine Privilege Levels により，ゲスト OS から隔離した領域で暗号鍵やログを管理する．本稿では，提案機構を COCONUT-SVSM 上に実現する方法，および評価した結果を述べる．評価では，外部の主体がログを検証することで，本研究で想定する攻撃を検知できること，および SVSM 呼び出しのオーバヘッドを測定により示す．この結果から，提案機構は実行時コストを伴うものの，外部の主体が受け取るログの信頼性がゲストオーナーに依存するという課題に対処できることを示す．

Implementation and Evaluation of a Mechanism for Trace-Log Provenance Verification and Tamper Detection in the Confidential-Information Dissemination Tracking on Confidential VMs

JURA INUI[1]　TOSHIHIRO YAMAUCHI[2]

Abstract: Although Confidential Virtual Machines (CVMs) can protect data in use, an entity that cannot log in to a CVM (hereafter, an external entity) cannot detect the replacement or tampering of logs by the CVM operator (hereafter, the guest owner). To address this issue, we propose a mechanism (hereafter, the proposed mechanism) that enables an external entity to verify the provenance of logs and detect log tampering. The proposed mechanism targets logs generated by Confidential-Information Dissemination Tracking on Confidential VMs. It uses the Virtual Machine Privilege Levels of AMD SEV-SNP to manage cryptographic keys and logs in a domain isolated from the guest OS. This paper describes how the proposed mechanism is implemented on COCONUT-SVSM and presents its evaluation results. The evaluation demonstrates that an external entity can detect the attacks considered in this study by verifying the logs and measures the overhead of SVSM calls. These results demonstrate that, although the proposed mechanism incurs runtime overhead, it addresses the problem that the trustworthiness of the logs received by an external entity depends on the guest owner.

1. はじめに

クラウド環境において，使用中のデータを保護する技術として Confidential Virtual Machine（以降，CVM）が注目されている．AMD Secure Encrypted Virtualization-Secure Nested Paging（以降，SEV-SNP）アーキテクチャを用いた CVM は，VM ごとに異なる暗号鍵を用いて VM のメモリを暗号化するとともに，悪意あるハイパーバイザによるデータのリプレイやメモリ再マッピングを防ぐ機構を備えている [1]．また，リモートアテステーションでは，CVM 起動時の構成の正当性を検証できる [2], [3].

本研究では，このような CVM の利用形態として，クラ

[1] 岡山大学大学院環境生命自然科学研究科
Graduate School of Environmental, Life, Natural Science and Technology, Okayama University, Okayama
[2] 岡山大学学術研究院環境生命自然科学学域
Faculty of Environmental, Life, Natural Science and Technology, Okayama University, Okayama

© 2026 Information Processing Society of Japan

This paper is work in progress and not peer-reviewed.

ウド環境で CVM を構築し，運用する主体と，CVM 外部から処理結果やログを確認する主体が異なる場合を想定する．本稿では，前者をゲストオーナー，後者を外部の主体と呼ぶ．例として，ゲストオーナーは，機密情報処理ワークロードのサービス提供者，外部の主体は，データ提供者や監査者が該当する．データ提供者は，ゲストオーナーが提供するワークロードに機密情報を提供し，処理結果を受け取る．この利用形態では，外部の主体は，CVM 内の機密情報の利用状況を確認する必要がある一方，CVM にログインする権限を持たない．このため，ゲストオーナーから提示されるログをもとに機密情報の利用状況を確認する必要がある．**課題：このログの信頼性はゲストオーナーに依存しており，悪意あるゲストオーナーによるログのすり替えや改ざんを検知できない．**

既存研究 [4] は，Virtual Machine Privilege Levels（以降，VMPLs）を用いてゲスト OS から隔離した領域でログを管理すること，およびログをセキュアチャネルを通じて遠隔の利用者に提供することを実現している．しかし，ログと CVM を対応付けて検証する仕組みは示されていないため，遠隔の利用者が，取得したログに基づいた由来確認や改ざん検知を行うことはできない．

そこで我々は，文献 [5] において，この課題に対処する上で求められる要件・要望を整理し，CVM 内で機密情報拡散追跡機能が生成するログ（以降，追跡ログ）を対象として，VMPLs を用いてログを保護し，外部の主体による追跡ログの由来確認と改ざん検知を可能にする機構（以降，提案機構）の設計を提案した．この設計では，VMPL1 で機密情報拡散追跡機能を含むゲスト OS を動作させ，VMPL1 より高い権限である VMPL0 で提案機構を含む Secure VM Service Module（以降，SVSM）を動作させる．提案機構は，追跡ログを VMPL0 内で受理したログ（以降，コミットログ）に対して，ハッシュ連鎖値の計算と署名の付与を行う．また，リモートアテステーションにより，コミットログへの署名に用いる暗号鍵（以降，PrivK_commit）が，アテステーション対象の CVM に由来することを確認可能にする．外部の主体は，受け取ったログについて，署名検証，ハッシュ連鎖値の確認，最新値の確認を行うことにより，対象の CVM に由来するログであること，および改ざんされていないことを確認可能である．

本稿では，この設計に基づく提案機構を，SEV-SNP を有効化した CVM 環境に実現する方法を示す．提案機構は，SVSM の実装の 1 つである COCONUT-SVSM [6] を拡張して実現する．また，提案機構とゲスト OS 間のデータのやり取りは，SVSM 呼び出しを活用することで実現する．評価では，CVM から提示されるログの改ざん，再送，すり替えという脅威に対して，外部の主体による検知が可能であることを示す．また，性能に関する評価では，機密情報拡散追跡機能がフックするシステムコールについて，提案機構追加によるオーバヘッドを定量的に示す．また，コミットログのデータ量に応じて，外部の主体によるコミットログの検証に要する時間がどの程度変化するかを示す．

2. 前提技術

2.1 Virtual Machine Privilege Levels

SEV-SNP には，VMPLs という仕組みがある [1]．VMPLs は，4 つの権限レベルが存在し，1 つの CVM 内に最大 4 層のアクセス権限の階層構造を構築できる．VMPL0 が最も高く，VMPL3 が最も低い権限レベルである．また，VMPL は，メモリページ単位で設定可能であり，低い VMPL で動作するソフトウェアから，高い VMPL が設定されたメモリページへアクセスすることはできない．一方，高い VMPL で動作するソフトウェアから，低い VMPL が設定されたメモリページへのアクセスや変更は可能である．

2.2 Secure VM Service Module

Secure VM Service Module（以降, SVSM）は, SEV-SNP により保護された CVM 内で，セキュリティ上重要な機能をゲスト OS から分離して実行するための環境である [7]．

SEV-SNP では，ハイパーバイザを信頼しないため，セキュリティ機能をハイパーバイザに委ねることができない．これに対し SVSM は，CVM 内の VMPL0 で動作し，ゲスト OS をより低い権限の VMPL で実行させる．この構成により，信頼しないホスト環境にセキュリティ上の重要なサービスを置くことなく，CVM 内部のゲスト OS から分離された領域に特権モジュールを構築できる．

SVSM のオープンソース実装の一つとして, COCONUT-SVSM [6] がある．COCONUT-SVSM は，GitHub 上で公開されており，ソースコードを改変して機能を追加できる．

2.3 リモートアテステーション

2.3.1 検証の対象

SEV-SNP を用いた CVM は，リモートアテステーションにより，アテステーション対象の CVM について，起動時の構成の正当性を外部から確認できる [2], [3]．

アテステーションレポート（以降，レポート）は，AMD Secure Processor（以降，ASP）による署名が付与されている．また，レポートには，CVM 起動時に計算される measurement に加え，VMPL，TCB version などの情報が含まれる．外部検証者は，レポートの署名検証により，当該レポートが正当な ASP により作成されたものであることを確認できる．また，外部検証者が保持する期待値とレポートの内容を比較することで，アテステーション対象の CVM が期待する構成で起動されたか否かを確認できる．

2.3.2 アテステーションレポートの拡張領域

SEV-SNP のリモートアテステーションで使用するレポートには，任意の 64bytes データを格納可能な領域（以降，

© 2026 Information Processing Society of Japan

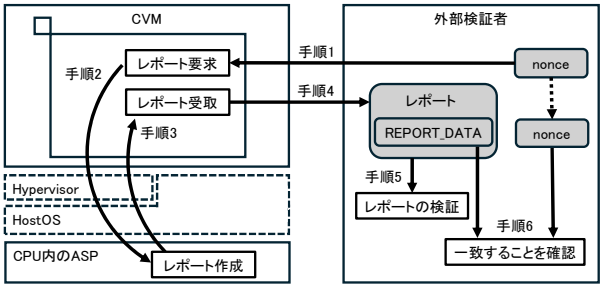


図 1: REPORT_DATA を活用したリモートアテステーション

REPORT_DATA）がある．REPORT_DATA を活用したリモートアテステーションの例を**図 1** に示し，以下で説明する．

（手順 1）外部検証者が 64bytes のランダムな nonce を生成し，アテステーション対象の CVM へ送信

（手順 2）CVM は，受信した nonce を REPORT_DATA に格納し，ASP へレポートを要求

（手順 3）ASP がレポートを作成し，CVM へ返却

（手順 4）CVM が受け取ったレポートを外部検証者へ送信

（手順 5）外部検証者が受信したレポートの署名を検証し，レポートに含まれる情報を確認

（手順 6）外部検証者が受信したレポートの REPORT_DATA に格納されている値と，（手順 1）で自身が生成した nonce が一致することを確認

（手順 6）により，外部検証者は，受信したレポートが当セッションのために生成されたことを確認できる．

2.4 機密情報拡散追跡機能

機密情報の拡散追跡機能 [8]（以降，機密情報拡散追跡機能）は，機密情報に対するファイル操作，プロセス間通信，および子プロセス生成に関するシステムコールをフックし，各処理を監視することで，計算機内における機密情報の拡散を追跡する機能である．本研究において，この機能を使用する理由は 2 つある．これを以下に示す．

（理由 1）デジタルフォレンジックに使用可能な追跡ログを生成可能

（理由 2）監視対象資源に対する処理に限定した情報追跡が可能

以上より，機密情報拡散追跡機能を用いることで，追跡ログから CVM 内の機密情報の拡散状況を調査できる．

3. CVM 上の機密情報拡散追跡機能における追跡ログの由来確認と改ざん検知機構

本章では，我々が文献 [5] で提案した機構について，想定する CVM の利用形態と課題，脅威モデル，および要件・要望と設計を概説する．

3.1 想定する CVM の利用形態と課題

本研究では，機密情報を所有する外部の主体が，リモートアテステーションにより CVM の起動時構成を確認した上で，CVM 上のワークロードに機密情報を提供する利用形態を想定する．このような利用形態では，クラウド環境で CVM を構築・運用し，ワークロードを提供する主体（ゲストオーナー）と，CVM のワークロードへ機密情報を提供する主体（外部の主体）が異なる場合がある．

この利用形態では，以下の 2 つの課題がある．

（課題 1）外部の主体は，CVM 内における機密情報の処理内容を直接調査できない

（課題 2）ゲストオーナーによって提示される機密情報の処理内容を示す情報が信頼できるとは限らない

（課題 1）に関して，この利用形態では，外部の主体は CVM にログインする権限を持たないため，CVM 内で機密情報がどのように扱われたかを，直接調査できない．そこで，本研究では，2.4 節に示した機密情報拡散追跡機能を CVM 内で動作させ，取得した追跡ログを外部の主体に提示することで（課題 1）に対処する．

一方，この方針では，（課題 2）が課題となる．外部の主体が受け取る追跡ログの信頼性がゲストオーナーに依存するため，外部の主体は，悪意あるゲストオーナーによるログのすり替えや改ざんを検知することができない．

3.2 脅威モデル

本研究における脅威は，外部の主体が，ゲストオーナーから提示されたログを調査する際，対象の CVM に由来しないログ，または改ざんされたログを正しいログとして受け入れることと定義する．

本研究では，SEV-SNP の保護機構，およびリモートアテステーションに関連する処理と検証結果を信頼する．加えて，リモートアテステーションの結果，CVM 起動時の measurement，VMPL，TCB version などが期待値と一致する場合，CVM 実行中，機密情報拡散追跡機能，および提案機構は正しく動作することを前提とする．

一方，クラウド事業者が管理者権限をもつホスト OS とハイパーバイザを信頼しない．SEV-SNP のメモリ暗号化により，CVM の保護対象となるメモリをホスト OS とハイパーバイザが平文で読むことはできない．また，CVM と外部の主体との通信路を信頼しない．ネットワークの通信路上に存在する主体は，送受信されるデータをすり替え，改ざん，および盗聴する可能性がある．

ゲストオーナーは，CVM 上のゲスト OS の管理者権限を持ち，外部の主体にログを提示する主体である．このため，対象の CVM に由来しないログの提示，または提示前のログの改ざんを行う可能性がある．また，ログや暗号鍵などの機密情報を意図せず漏えいさせる可能性がある．しかし，SEV-SNP の VMPLs は正しく機能し，ゲストオーナーはゲスト OS よりも高い VMPL が設定されたメモリページへアクセスできない．

© 2026 Information Processing Society of Japan

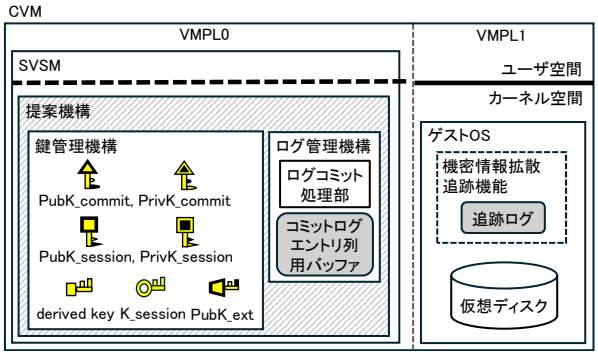


図 2: CVM と提案機構の構成

本研究の保護対象は，提案機構が VMPL0 内で受理した後のコミットログである．一方，機密情報拡散追跡機能が追跡ログを生成した後，提案機構へログを渡す前に行われる改ざんや削除は，本研究の保護対象外とする．また，CVM や追跡ログ生成の停止，通信の遮断といった可用性に関する攻撃，保存データのロールバック攻撃，サイドチャネル攻撃，物理的な攻撃，および SEV-SNP の脆弱性を利用する攻撃は対象外とする．

3.3 要件・要望と設計

脅威モデルを満たし，（課題 2）に対処するためには，以下の 4 つの要件を満たす必要がある．

（要件 1）提案機構はゲストオーナーがアクセス可能な領域から分離すること

（要件 2）コミットログを秘匿して外部の主体へ提示できること

（要件 3）コミットログの由来を確認できること

（要件 4）コミットログの改ざんを検知できること

また，CVM をシャットダウンした場合，シャットダウン前のコミットログが喪失し，当該コミットログを外部の主体に提示できない．これに対し，以下の要望がある．

（要望 1） CVM 再起動後，シャットダウン前のコミットログを継続して利用可能であること

上記の要件・要望を満たすよう設計した提案機構の構成を**図 2** に示す．図 2 に示すように，ゲスト OS と機密情報拡散追跡機能を VMPL1 で動作させ，SVSM と提案機構を VMPL0 で動作させる．提案機構は，鍵管理機構とログ管理機構から構成され，VMPL1 で生成された追跡ログを受理し，ハッシュ連鎖値，および PrivK_commit による署名を付与したコミットログエントリを管理する．また，コミットログの提示時は，最新のハッシュ連鎖値（以降，`Latest`）に PrivK_commit による署名を付与した署名済み `Latest` とコミットログエントリ列をまとめた提示データを，外部の主体との間で確立したセッション共通鍵（以降，K_session）により暗号化する．

外部の主体は，リモートアテステーションによって提案機構の構成，署名検証用のパブリックキー（以降，PubK_commit），およびセッション確立用のパブリックキー（以降，PubK_session）の由来を確認する．また，PubK_session を用いて CVM との間で確立した K_session により，CVM から受信した暗号化済み提示データを復号する．さらに，各ログの署名検証，ハッシュ連鎖値の確認，および `Latest` の検証を行うことで，ログの由来とコミット後の改ざんを確認できる．

CVM のシャットダウン時には，コミットログエントリ列と PrivK_commit を derived key [3] で暗号化して仮想ディスクに保存する．再起動時，前回起動時と同じ derived key を再導出し，暗号化したデータを復号する．

4. 提案機構の実現方法

4.1 対処する課題

3.3 節で述べた設計に基づいて提案機構を実現するためには，以下の課題に対処する必要がある．

（実現課題 1）ゲスト OS から提案機構への追跡ログの受け渡し

（実現課題 2）提案機構からゲスト OS への提示データの受け渡し

以降の節で，上記の実現課題に対処する方法を述べる．

4.2 ゲスト OS から提案機構への追跡ログの受け渡し

VMPL1 で動作するゲスト OS は，VMPL0 のメモリページにアクセスできないため，生成した追跡ログを提案機構に受け渡すためには，VMPL 間の制御移行とともに追跡ログを受け渡す必要がある．

この課題へ対処するため，AMD が定める SVSM 呼び出し規約 [7] を用いた．この規約では，各 vCPU に対応する 4KiB の Calling Area とレジスタを介して，要求および応答を受け渡す．具体的には，ゲスト OS は，要求内容を Calling Area およびレジスタへ格納した後，Calling Area の要求フラグを設定して VMGEXIT を実行する．VMPL0 に制御移行した後，SVSM は要求に応じた処理を行う．

提案機構では，当規約に従い，COCONUT-SVSM の SVSM 呼び出し処理を拡張した．具体的には，既存の SVSM プロトコル番号とは異なるプロトコル番号を追加し，処理内容を識別する Call ID を割り当てた．ゲスト OS は，Calling Area に追跡ログを書き込み，追跡ログ通知用の Call ID を指定して，SVSM 呼び出しを行う．VMPL0 で動作する提案機構は，RAX レジスタからプロトコル番号と Call ID を取得し，Calling Area から追跡ログを取得する．これにより，ゲスト OS から提案機構への追跡ログの受け渡しを実現した．

4.3 提案機構からゲスト OS への提示データの受け渡し

CVM 内のゲスト OS は，外部の主体によるログ要求を受信した際，SVSM 呼び出しを行って提案機構から暗号化

© 2026 Information Processing Society of Japan

表 1: CVM の構成

ホスト	CPU	AMD EPYC 9124 16-Core Processor
	OS	Ubuntu Server 24.04.3 LTS
	カーネル	Linux kernel 6.11.0
	VMM	QEMU 8.2.2
ゲスト	vCPU 数	12
	メモリ	8192 MiB
	SVSM	COCONUT-SVSM
	OS	Ubuntu Server 24.04.3 LTS
	カーネル	Linux kernel 6.11.0

済み提示データを受け取る．この際，暗号化済み提示データ量が，Calling Area の 4KiB を超える可能性がある．

この課題への対処方針として，以下の 2 つの方式がある．

（方式 1）暗号化済み提示データを分割し，SVSM 呼び出しを必要な回数だけ繰り返す方式

（方式 2）暗号化済み提示データを保持しているメモリページに，VMPL1 の読み取り権限を付与する方式

（方式 1）では，コミットログのデータ量に応じて SVSM 呼び出しの回数が増加する．一方，VMPL0 と VMPL1 間のデータのやり取りに，既存の Calling Area を利用できる．

（方式 2）では，提案機構が，ゲスト OS に暗号化済み提示データを格納したメモリページの先頭アドレスを提示する．これにより，SVSM 呼び出し回数を削減できる．一方，提案機構が管理するメモリページに，VMPL1 の読み取り権限を与える必要がある．このため，VMPL0 のメモリページが新たな VMPL 間のインタフェースとなり，提案機構が管理する対象が増加する．

本研究では，提案機構追加による SVSM の TCB 増加を低減するため，既存の VMPL 間インタフェースを使用する（方式 1）で，提示データの受け渡しを実現した．

5. 評価

5.1 評価方針

本稿における評価では，以下の 3 つの RQ に回答する．

RQ1：外部の主体は，CVM から受信したコミットログが，アテステーションした CVM に由来しないこと，または改ざんされたことを検知可能か．

RQ2：提案機構は，機密情報拡散追跡機能がフックするシステムコールの実行時間や CVM の起動時間に，どの程度のオーバヘッドを与えるか．

RQ3：提示データの受け渡しに要する時間，および外部の主体によるコミットログの検証に要する時間は，データ量に応じてどのように変化するか．

5.2 評価環境

評価に用いる実験環境を**表 1**，および**表 2** に示す．表 1 には，提案機構を実現した CVM の構成を示している．使用している Linux kernel 6.11.0 は，COCONUT-SVSM 対応パッチを適用したものである．

表 2: 外部の主体が使用する環境（VM）の構成

ホスト	CPU	Intel(R) Core(TM) i5-9600K
	OS	Windows 11 Home
	仮想化ソフトウェア	VirtualBox 7.2.4
ゲスト	vCPU 数	6
	メモリ	16384 MiB
	OS	Ubuntu 24.04.3 LTS
	カーネル	Linux kernel 6.15.3

評価結果のばらつきを低減するため，表 1 の環境では，Core Performance Boost（CPB）と同時マルチスレッディングを無効化し，ホスト OS の CPU 周波数制御の上限値および下限値を，いずれも 3GHz に設定した．また，表 2 の環境では，Intel Turbo Boost Technology を無効化した．

両計算機は，同じ VLAN に接続しており，CVM，および外部の主体が使用する VM は，いずれもホスト OS の NAT 機能を介してネットワークに接続している．また，CVM は，外部の主体に HTTPS 用の TCP ポートのみ公開しており，VM から CVM にログインすることはできない．

なお，本評価では，コミットログと `Latest` の署名を ECDSA-P384 with SHA-384，ハッシュ連鎖値を SHA-384，暗号化を AES-256-CTR で実現した提案機構を用いた．

5.3 外部の主体による検証可能性の評価（RQ1）

5.3.1 評価項目

本評価に先立ち，外部の主体から CVM にログ要求を行い，検証を通して全てのコミットログを取得できることを確認した．その上で，悪意あるゲストオーナーによる暗号化済み提示データへの操作に対して，外部の主体が正当ではない提示データの受理を防止できるか否かを実験により検証した．

ここでは，以下の 3 つの攻撃を想定し，評価を行った．

（攻撃 1）暗号化済み提示データの改ざん

（攻撃 2）過去の暗号化済み提示データの再送

（攻撃 3）別 CVM 生成の暗号化済み提示データへのすり替え

5.3.2 暗号化済み提示データの改ざん

本攻撃は，ゲストオーナーが，送信前の暗号化済み提示データに対して改ざんを行う攻撃である．本実験では，事前に 314 件のコミットログエントリを生成した．また，暗号化済み提示データの送信処理を行うプログラムを改変し，送信前の暗号化済み提示データに対して，中央付近のデータを改ざんする処理を追加した．

外部の主体から CVM にログ要求を行い，得られた暗号化済み提示データを検証した．結果として，第 1 エントリから第 156 エントリまでの検証に成功し，ログを取得できた．また，第 157 エントリにおいて改ざんを検知し，不正

© 2026 Information Processing Society of Japan

なログと判断できた．

したがって，外部の主体は，暗号化済み提示データの改ざんを検知できる．また，改ざんされていないログは，検証済みログとして取得できる．

5.3.3 過去の暗号化済み提示データの再送

本攻撃は，過去のログ要求時に送信した暗号化済み提示データを，別のログ要求時に再送する攻撃である．本実験では，1 回目のログ要求時に外部の主体へ送信した暗号化済み提示データを保存し，2 回目のログ要求に対して，保存した暗号化済み提示データを再送した．

結果として，1 回目のログ要求時に取得した暗号化済み提示データは，全てのログの検証に成功した．一方，2 回目のログ要求時に取得した暗号化済み提示データは，`Latest` の署名検証に失敗した．また，コミットログエントリは，ログ長が不正であるとして検証に失敗した．この原因は，暗号化に用いる K_session がログ要求ごとに生成されるため，2 回目に受け取った暗号化済み提示データを正しく復号できないからである．

したがって，過去の暗号化済み提示データを再送された場合，外部の主体は，不正なログと判断できる．

5.3.4 別 CVM 生成の暗号化済み提示データへのすり替え

本攻撃は，ゲストオーナーが，CVM から送信する暗号化済み提示データを，別の CVM が生成した暗号化済み提示データにすり替える攻撃である．本実験では，表 1 に示したホスト環境に，CVM1 と CVM2 を構築した．また，外部の主体から CVM1 に対してログ要求を行い，CVM1 内の提案機構から暗号化済み提示データを取得して CVM2 に渡した．CVM2 は，外部の主体によるログ要求に対し，この暗号化済み提示データを送信するよう設定した．

外部の主体から CVM2 にログ要求を行い，得られた暗号化済み提示データを検証した．結果として，`Latest` の署名検証に失敗した．また，コミットログエントリは，ログ長が不正であるとして検証に失敗した．この原因は，5.3.3 項と同じである．K_session が今回のログ要求に対応していないため，正しく復号できず，検証に失敗した．

したがって，別 CVM 生成の暗号化済み提示データへのすり替えに対し，外部の主体は，不正なログと判断できる．

5.4 提案機構によるオーバヘッド（RQ2）

5.4.1 システムコールの実行時間の評価

提案機構がシステムコールの実行時間に与えるオーバヘッドを評価するため，機密情報拡散追跡機能がフックする 19 種類のシステムコールの実行時間を測定した．

測定は，提案機構追加前のカーネルと追加後のカーネルで行った．各カーネルにおいて，測定対象のシステムコールを 10,000 回実行し，各試行で測定した TSC 値から実行時間を求めた．提案機構追加後のカーネルでは，評価用プロセスを管理対象プロセスにしたうえで測定を行った．

表 3: ファイル操作と子プロセス生成に関するシステムコールのオーバヘッド（µs）

システムコール	機構追加前	機構追加後	オーバヘッド
open(file)	2.98	811.04	808.06
read(file)	2.67	808.92	806.25
write(file)	0.64	834.28	833.64
close(file)	1.47	806.08	804.61
fork	49.10	952.59	903.49
rename	17.05	832.82	815.77

各カーネルにおける測定結果を，**表 3** と付録の**表 A·1** に示す．また，各試行の測定結果の分布を付録の**図 A·1** に示す．表 3 および付録の表 A·1 には，10,000 回の試行で取得した TSC 値のうち，中央値を用いて求めた各システムコールの実行時間を示している．また，オーバヘッドは，提案機構追加後と追加前の実行時間の差である．

各システムコールのオーバヘッドから，パイプに対する `open` と `write`，`msgrcv`，および `shmat` は，およそ 1,610µs である．一方，これらを除く 15 種類のシステムコールは，およそ 800µs から 900µs のオーバヘッドである．この結果は，システムコール実行中の SVSM 呼び出しの回数に起因する．前者は，ログ生成を行う部分が 2 箇所あるため，SVSM 呼び出しが 2 回行われている．一方，後者は，ログ生成を行う部分が 1 箇所であり，SVSM 呼び出しの回数が 1 回である．このため，前者に後者のおよそ 2 倍のオーバヘッドが生じている．

5.4.2 SVSM 呼び出し時の処理ごとの実行時間

追跡ログ生成直後に行う SVSM 呼び出しでは，主に，VMPL の切り替え，ハッシュ連鎖値の計算，および署名の付与といった処理が行われる．本評価では，`open` システムコールフック時の追跡ログ生成後，SVSM 呼び出し前後の TSC 値を測定し，実行時間を求めた．また，SVSM 内部で行われる各処理の実行時間を測定した．

各処理に要する実行時間を**表 4** に示す．表 4 には，`open` システムコールを 10,000 回実行した際に取得した TSC 値のうち，中央値を用いて求めた実行時間を示している．

表 4 から，署名の付与に要する時間は SVSM 呼び出し全体のおよそ 98%を占めている．VMPL 切り替え処理は，14.64µs を要するものの，署名の付与に要する実行時間と比較すると十分小さい．また，表 3 に示した `open` システムコールのオーバヘッド，表 4 に示した SVSM 呼出処理全体の中央値から，`open` システムコールのオーバヘッドのおよそ 99%を SVSM 呼び出し処理が占めている．

5.4.3 提案機構追加後の CVM の起動時間

derived key と，PubK_commit，PrivK_commit（以降，commit 鍵対）の生成処理が，CVM の起動時間に与える影響を評価した．本評価では，シャットダウン時にデータの

© 2026 Information Processing Society of Japan

表 4: SVSM 呼出時の処理時間とオーバヘッドへの影響

処理名	中央値（µs）	SVSM 呼出時の処理時間に対する割合（%）
SVSM 呼出処理全体	800.08	100
VMPL 切り替え処理	14.64	1.83
ハッシュ連鎖値の計算	0.68	0.08
署名の付与	783.51	97.92

表 5: derived key と commit 鍵対生成処理に要する時間

処理名	中央値（µs）	CVM の起動時間に対する割合（%）
CVM 起動処理全体	20089415.56	100
derived key 生成	1616.09	0.008
commit 鍵対生成	1392.34	0.006

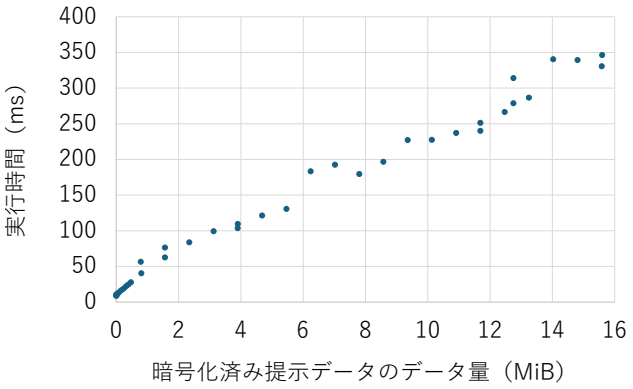


図 3: 提示データの受け渡しに要する時間の変化

保存を行わず，起動のたびに新しい commit 鍵対を生成するように設定し，提案機構を実現した CVM の起動時間全体と，各処理に要する時間を測定した．

10 回の試行を行い，得られた結果を**表 5** に示す．表 5 の時間は，CVM 起動処理全体に要する時間と比較して十分に小さく，derived key の生成時間は 0.008%，commit 鍵対の生成時間は 0.006%のみである．したがって，これらの処理が CVM の起動時間に与える影響は極めて小さい．

5.5 データ量に応じた実行時間の変化に関する評価（RQ3）

5.5.1 提示データの受け渡しに要する時間の変化

データ量に応じた提示データの受け渡しに要する時間の変化を評価した．本評価では，提案機構がログ要求を受けてから提示データを暗号化し，暗号化済み提示データをゲスト OS へ渡し終えるまでの時間を測定した．

データ量に応じた実行時間の変化を**図 3** に示す．図 3 から，データ量増加に伴い，受け渡しに要する時間はおおむね線形に増加している．また，測定した範囲内では，実行時間が急激に増大する傾向は見られない．

したがって，4.3 節の（方式 1）による提示データの受け渡しは，データ量に応じて予測可能な実行時間を示す．

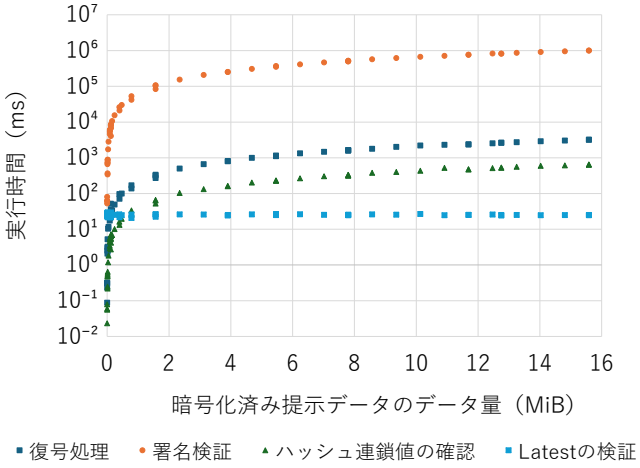


図 4: コミットログ検証時の処理ごとの実行時間の変化

5.5.2 コミットログの検証処理に要する時間の変化

本項では，外部の主体がコミットログを検証する際の各処理に要する時間が，データ量に応じてどの程度変化するかを示す．コミットログの検証では，提示されたデータの復号，署名検証，ハッシュ連鎖値の確認，および Latest の検証を行う．本評価では，表 2 に示した環境で各処理に要する実行時間を測定した．

データ量に応じた各処理に要する実行時間の変化を**図 4** に示す．図 4 では，実行時間の差を明確にするため，縦軸を底 10 の対数とした．図 4 から，データ量増加に伴い，復号処理，署名検証，およびハッシュ連鎖値の確認に要する時間が増加している．一方，Latest の検証に要する時間は，データ量に依存せず一定である．

署名検証に要する時間が最も大きく，データ量の増加に伴う処理時間の増加量も最も大きい．したがって，外部の主体によるコミットログの検証処理では，署名検証が全体の検証時間の増加に最も大きく影響している．

6. 考察

6.1 想定する利用形態への適用可能性

RQ1 に対する評価において，暗号化済み提示データの改ざん，再送，および別 CVM が生成した提示データへのすり替えに対し，外部の主体は検証により，不正なログの受理を防止できることを確認した．一方で，性能面について，システムコールのオーバヘッド，および外部の主体による検証に要する時間は大きい．

したがって，提案機構は，3.1 節で示した（課題 2）に対処でき，実行時コストを許容できる用途に適用可能である．

6.2 提案機構の制約

提案機構には，2 つの制約がある．

1 つ目は，提案機構の保証範囲が，VMPL0 内で追跡ログを受理した後に限定されることである．機密情報拡散追跡機能がログを生成した後，提案機構へ渡すまでに行われ

© 2026 Information Processing Society of Japan

る改ざんは防止できない．このため，CVM 実行中，機密情報拡散追跡機能は正しく動作することが必要である．

2 つ目は，保持可能なコミットログ量に上限があることである．今回の実装では，VMPL0 内で保持するデータの最大サイズを 16MiB に設定しているため，保持できるコミットログ量はこの範囲に制限される．このため，ログ生成量が多いワークロードや CVM を長期間運用する用途では，定期的にログ要求を行い，提示済みのコミットログを VMPL0 内の保持データから削除することで，保持量が上限に達することを防ぐ必要がある．

7. 関連研究

SEV-SNP の VMPLs を用いて，ログを保護する研究として，Veil [4] がある．Veil は，ゲスト OS が生成したログをゲスト OS よりも高い VMPL で保持し，セキュアチャネルを介してリモートユーザへ送信する仕組みを実現している．Veil では，ログをゲスト OS よりも高い VMPL で動作する送信機構から直接取得することで，ログの信頼性を確保する．一方，本研究では，ログの由来と改ざん有無を確認可能にすることで，ログの信頼性を確保する．

CVM の内部状態を遠隔から調査可能にする研究として，00SEVen [9] がある．00SEVen は，ゲスト OS よりも高い VMPL でエージェントを動作させ，CVM のメモリやレジスタ状態を調査する仕組みを実現している．一方，調査対象がメモリやレジスタ状態であり，CVM 内で行われる機密情報に対する操作を調査可能にする本研究とは異なる．

TEE 外の主体と TEE との間で安全なチャネルを確立する研究として，Separate but Together [10] がある．Separate but Together は，TLS の証明書とリモートアテステーションを対応付け，確立したチャネルがアテステーション対象の TEE で終端していることを保証する仕組みを実現している．リモートアテステーションにパブリックキーを対応付ける点で本研究と共通している一方，対応付けたパブリックキーを通信チャネルの確立に用いており，ログの検証に用いる本研究とは異なる．

TEE を用いたログの改ざん検知を目的とする研究として，CUSTOS [11] がある．CUSTOS は，OS ログを対象として，ハッシュ値計算と署名付与により，ログ完全性の第三者検証を可能にしている．一方，Intel SGX 上に実現されており，対象とするアーキテクチャが本研究とは異なる．

8. おわりに

本稿では，文献 [5] で述べた CVM 上の機密情報拡散追跡機能が生成する追跡ログの由来確認と改ざん検知を外部から可能にする機構（以降，提案機構）の設計をもとに，提案機構を COCONUT-SVSM 上に実現し，評価した結果を述べた．評価の結果，悪意あるゲストオーナーによる暗号化済み提示データの改ざん，再送，およびすり替えを，外部の主体が検知できることを確認した．また，暗号化済み提示データが一部改ざんされた場合でも，改ざん箇所より前のログを検証し，取得可能であることを示した．一方，システムコールの実行時に，SVSM 呼び出し 1 回あたり 800µs のオーバヘッドが生じ，この主因が，署名の付与であることを確認した．

以上のことから，提案機構は，実行時コストを伴うものの，外部の主体が受け取るログの信頼性がゲストオーナーに依存するという課題に対処できる．

今後の課題として，システムコール実行時のオーバヘッドの改善やコミットログの検証に要する時間の改善がある．

謝辞 本研究の一部は，JSPS 科研費 25K03119 による助成を受けたものです．

参考文献

[1] Advanced Micro Devices, Inc.: AMD SEV-SNP: Strengthening VM Isolation with Integrity Protection and More, Publication No. 70366 (2020).

[2] Advanced Micro Devices, Inc.: AMD SEV-SNP Attestation: Establishing Trust in Guests, available from ⟨https://www.amd.com/content/dam/amd/en/documents/developer/lss-snp-attestation.pdf⟩ (accessed 2026-05-19) (2022).

[3] Advanced Micro Devices, Inc.: SEV Secure Nested Paging Firmware ABI Specification, Publication No. 56860, Rev. 1.58 (2025).

[4] Ahmad, A., Ou, B., Liu, C. et al.: Veil: A Protected Services Framework for Confidential Virtual Machines, *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, pp. 378–393 (online), DOI: 10.1145/3623278.3624763 (2023).

[5] 乾 朱来, 山内利宏: Confidential VM 上の機密情報拡散追跡機能における追跡ログの由来確認と改ざん検知機構の設計, 情報処理学会 マルチメディア，分散，協調とモバイル (DICOMO2026) シンポジウム論文集, Vol. 2026, No. 1, pp. 1553–1560 (2026).

[6] coconut-svsm: COCONUT Secure VM Service Module, available from ⟨https://github.com/coconut-svsm/svsm⟩ (accessed 2026-05-20).

[7] Advanced Micro Devices, Inc.: Secure VM Service Module for SEV-SNP Guests, Publication No. 58019, Rev. 1.01 (2026).

[8] 田端利宏, 箱守 聰, 大橋 慶, 植村晋一郎, 横山和俊, 谷口秀夫: 機密情報の拡散追跡機能による情報漏えいの防止機構, 情報処理学会論文誌, Vol. 50, No. 9, pp. 2088–2102 (2009).

[9] Schwarz, F. and Rossow, C.: 00SEVen – Re-enabling Virtual Machine Forensics: Introspecting Confidential VMs Using Privileged in-VM Agents, *Proceedings of the 33rd USENIX Security Symposium*, pp. 1651–1668 (2024).

[10] Weinhold, C., Sardar, M. U., Mihalcea, I. et al.: Separate but Together: Integrating Remote Attestation into TLS, *Proceedings of the 2025 USENIX Annual Technical Conference*, pp. 1319–1326 (2025).

[11] Paccagnella, R., Datta, P., Hassan, W. U. et al.: CUSTOS: Practical Tamper-Evident Auditing of Operating Systems Using Trusted Execution, *Proceedings of the Network and Distributed Systems Security Symposium*, (online), DOI: 10.14722/ndss.2020.24065 (2020).

© 2026 Information Processing Society of Japan

付　　録

A.1　システムコールのオーバヘッド

表 A·1: プロセス間通信に関するシステムコールのオーバヘッド（µs）

システムコール	機構追加前	機構追加後	オーバヘッド
send(socket)	1.76	808.90	807.14
recv(socket)	1.72	807.92	806.20
close(socket)	3.25	810.77	807.52
open(pipe)	3.36	1613.09	1609.73
read(pipe)	1.08	805.94	804.86
write(pipe)	1.11	1606.94	1605.83
close(pipe)	1.69	806.90	805.21
msgsnd	1.28	805.06	803.78
msgrcv	1.27	1609.36	1608.09
msgctl	0.94	807.38	806.44
shmat	5.07	1622.43	1617.36
shmdt	4.41	816.10	811.69
shmctl	0.93	804.47	803.54

A.2　提案機構追加前と追加後のシステムコールの処理時間

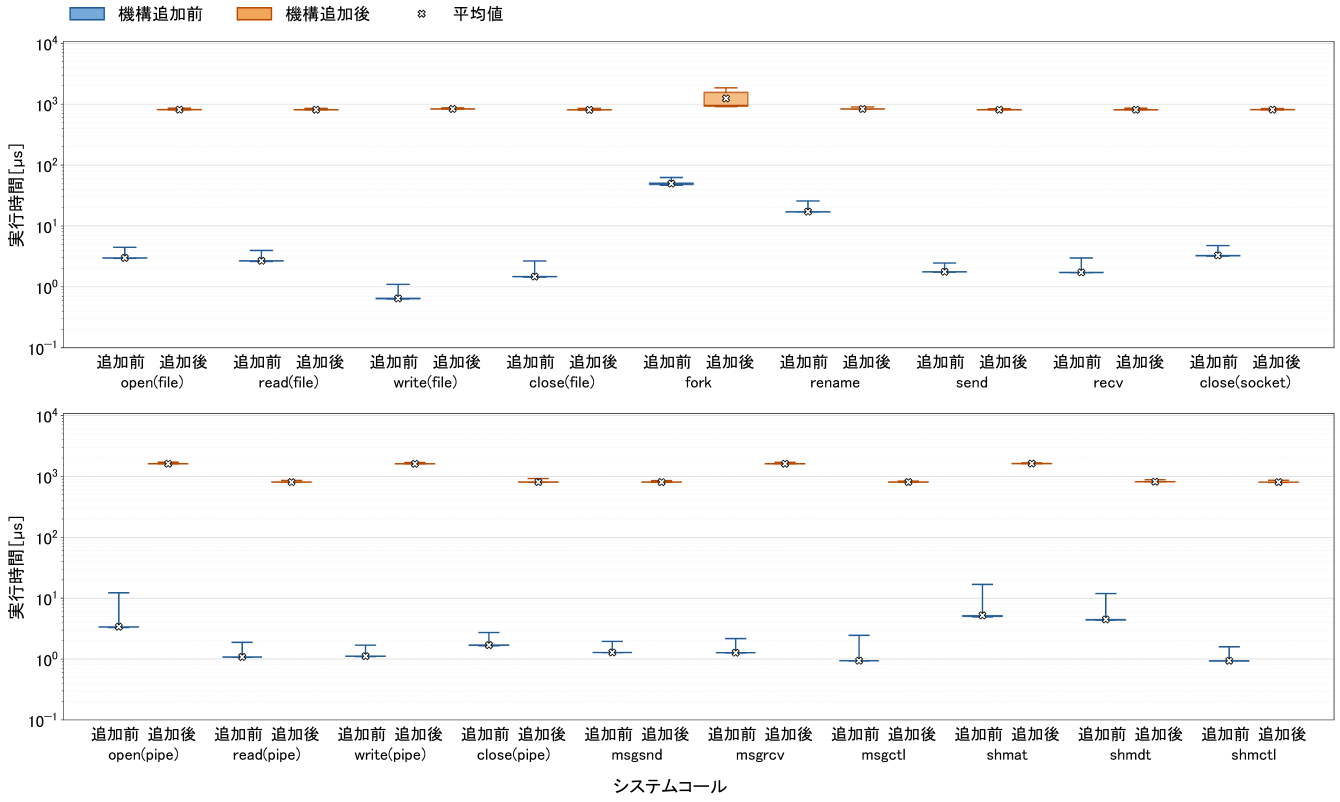


図 A·1: 提案機構追加前と追加後のシステムコールの処理時間

© 2026 Information Processing Society of Japan